

Chip in, big win

UVM Driver 常考题 / 真题汇总

Part 3 / 3 · 题目 25 ~ 34

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 25-34：Pipeline 续与多笔在途（题25-27）、调试与代码修复（题28-31）、覆盖率与高级特性（题32-34）。

题目 25 乱序响应：按 ID 路由 get_response

难度：高级

标签：out-of-order、get_response(id)、重排、响应队列

【题目描述】

实现乱序响应场景：sequence 批量发出多笔请求，driver 按到达顺序回传响应，sequence 按 id 取回对应响应。

设计要点：

1. driver 两线程分离：process_trans（get() 取 item 发请求，不等响应）和 recv_responses（监听接口，按 id 回传）；

2. driver 回传时必须正确设置 sequence_id（从原始 req 获取），sequencer 才能按 id 路由到正确的 sequence；

3. seq_id_map 关联表：维护 id -> sequence_id 的映射，响应线程据此正确设置回传 rsp 的 sequence_id；

4. sequence 端：先批量 finish_item 快速发出所有请求，再批量 get_response(rsp, transaction_id) 按 id 取回；

5. 响应队列由 UVM sequencer 内部管理，sequence 取完后队列自然清空，不会无限增长。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int seq_id_map[int]; // id -> sequence_id
    semaphore map_sem;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        map_sem = new(1);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            recv_responses();
            monitor_reset();
        join
    endtask

```

```
    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    // 请求线程：get() 取 item，发请求，登记 seq_id
    task process_trans();
        my_transaction req;
        forever begin
            seq_item_port.get(req);
            map_sem.get();
            seq_id_map[req.id] = req.get_sequence_id();
            map_sem.put();
            drive_item(req);
        end
    endtask

    // 响应线程：监听接口，按 id 路由回传
    task recv_responses();
        my_transaction rsp;
        forever begin
            do @(v_if.drive_mp.drive_cb);
            while (!v_if.drive_cb.rsp_valid);
            begin
                int rsp_id = v_if.drive_cb.rsp_id;
                rsp = my_transaction::type_id::create("rsp");
                rsp.id = rsp_id;
                rsp.data = v_if.drive_cb.rsp_data;
                map_sem.get();
                if (seq_id_map.exists(rsp_id)) begin
                    rsp.set_sequence_id(seq_id_map[rsp_id]);
                    seq_id_map.delete(rsp_id);
                end else
                    `uvm_error("DRV", $sformatf("Unknown rsp id=%0d", rsp_id))
                map_sem.put();
                seq_item_port.put_response(rsp);
            end
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
```

```
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass

// === sequence 端最小接收骨架（乱序 get_response）===
// class my_seq extends uvm_sequence#(my_transaction);
// task body();
// my_transaction reqs[4], rsp;
// foreach (reqs[i]) begin
// reqs[i] = my_transaction::type_id::create($sformatf("req%0d",i));
// reqs[i].id = i;
// start_item(reqs[i]); finish_item(reqs[i]);
// end
// foreach (reqs[i])
// get_response(rsp, reqs[i].get_transaction_id());
// endtask
// endclass
```

【模版套用说明】

driver 维护 seq_id_map（id -> sequence_id），响应线程按 id 查表设置 sequence_id，put_response 由 sequencer 路由到正确 sequence。sequence 侧批量 finish_item 后批量 get_response(id) 取回，实现真正的乱序。

题目 26 Split-phase：请求线程与完成线程解耦

难度：高级

标签：split-phase、并发线程、mailbox、reset 中止

【题目描述】

实现请求和完成完全解耦的 split-phase driver。

两个线程的职责划分：

1. req_thread：从 sequencer 取 item（get_next_item），发起接口请求，把 req 登记到 pending_mb mailbox，调用 item_done（不带 rsp）；

2. cpl_thread：阻塞等待接口完成信号（rsp_valid），从 pending_mb 取出对应上下文，构造 rsp，通过 put_response 回传；

3. mailbox 天然线程安全（内建 blocking get/put），无需额外 semaphore；

4. reset 中止：monitor_reset 在 mailbox 中 drain 所有挂起项目，防止 cpl_thread 使用到无效的旧上下文；

5. mailbox 深度无限（new(0)），天然支持 multi-outstanding，cpl_thread 按 FIFO 顺序处理。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    mailbox #(my_transaction) pending_mb;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        pending_mb = new(0);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            req_thread();
            cpl_thread();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            // drain mailbox，防止 cpl_thread 使用旧上下文
```

```
            begin
                my_transaction tmp;
                while (pending_mb.try_get(tmp)) ;
            end
            v_if.drive_cb.data <= 8'd0;
            `uvm_info("DRV", "Reset: mailbox drained, bus idle", UVM_LOW)
        end
    endtask

    // 请求线程：取 item -> 发请求 -> 登记 -> item_done
    task req_thread();
        my_transaction req;
        forever begin
            seq_item_port.get_next_item(req);
            `uvm_info("DRV",
                $sformatf("REQ: id=%0d [registered]", req.id), UVM_HIGH)
            write_item(req);
            pending_mb.put(req);
            seq_item_port.item_done();
        end
    endtask

    // 完成线程：等完成信号 -> 取上下文 -> 回传
    task cpl_thread();
        my_transaction req, rsp;
        forever begin
            do @(v_if.drive_mp.drive_cb);
            while (!v_if.drive_cb.rsp_valid);
            pending_mb.get(req);
            `uvm_info("DRV",
                $sformatf("CPL: id=%0d [completed]", req.id), UVM_HIGH)
            $cast(rsp, req.clone());
            rsp.set_sequence_id(req.get_sequence_id());
            rsp.data = v_if.drive_cb.rsp_data;
            seq_item_port.put_response(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
```

```
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

mailbox 作为 req/cpl 两线程的同步通道，天然线程安全无需 semaphore。monitor_reset 负责 drain mailbox 和拉 idle，防止 cpl_thread 用到旧上下文。日志打印 [registered]/[completed] 两个环节，调试链路完整。

题目 27 Driver 侧 Credit/Flow-Control 管理

难度：高级

标签：credit、flow-control、节流、同步更新、缓存策略

【题目描述】

实现带 credit 流控的 driver，使发送速率受 DUT 动态授信控制。

Credit 机制设计：

1. credit_cnt 变量维护当前可用 credit 数，初始值 init_credit 由配置决定；

2. update_credit 线程：在 @posedge clk 同步边沿采样 credit_inc，累加到 credit_cnt（同步采样避免与 driver 驱动产生竞态）；

3. process_trans 线程：发送前检查 credit_cnt，为 0 时 wait(credit_avail) 阻塞（节流），credit 到来时被 event 唤醒；

4. 发送成功后 credit_cnt-- 消耗一个 credit，打印剩余 credit；

5. semaphore credit_sem 保护 credit_cnt 的读写，防止两线程同时修改；

6. reset 时重置 credit 为 init_credit。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int credit_cnt = 0;
    int init_credit = 2;
    semaphore credit_sem;
    event credit_avail;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        credit_sem = new(1);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
        void'(uvm_config_db#(int)::get(this, "", "init_credit", init_credit));
        credit_cnt = init_credit;
        if (credit_cnt > 0) -> credit_avail;
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            update_credit();
            monitor_reset();
        join
```

```
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
            credit_sem.get();
            credit_cnt = init_credit;
            if (credit_cnt > 0) -> credit_avail;
            credit_sem.put();
        end
    endtask

    // Credit 更新线程：同步采样
    task update_credit();
        forever begin
            @(posedge v_if.mon_cb.clk);
            if (v_if.mon_cb.credit_inc > 0) begin
                credit_sem.get();
                credit_cnt += v_if.mon_cb.credit_inc;
                `uvm_info("DRV",
                    $sformatf("Credit +%0d -> total=%0d",
                              v_if.mon_cb.credit_inc, credit_cnt), UVM_HIGH)
                -> credit_avail;
                credit_sem.put();
            end
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            // 节流：无 credit 时等待唤醒
            while (credit_cnt == 0) begin
                `uvm_info("DRV", "No credit, throttling...", UVM_MEDIUM)
                @credit_avail;
            end
            seq_item_port.get_next_item(req);
            credit_sem.get();
            credit_cnt--;
            `uvm_info("DRV",
                $sformatf("Credit consumed: remaining=%0d", credit_cnt), UVM_HIGH)
            credit_sem.put();
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
```

```
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

update_credit 在 @posedge clk 同步边沿更新，避免与 drive 信号竞争。credit_sem 保护计数器，event credit_avail 作为 process_trans 的唤醒机制。while + @event 实现高效节流（不自旋）。

题目 28 修复：缺失 item_done 导致 finish_item Hang

难度：中级

标签：hang、item_done 配对、异常分支、结构化修复、断言

【题目描述】

修复因分支路径漏掉 item_done() 导致 sequence 永久阻塞在 finish_item 的 bug。

Bug 根因分析：

原始 driver 在 read_item 检测到 rsp_err==1 时直接 return（早退），未到达 process_trans 末尾的 item_done，

导致 sequencer 永远等不到 item_done，finish_item 无限阻塞（hang）。

修复策略（结构化，不粗暴）：

1. 把 item_done 的调用提升到 process_trans 循环末尾统一执行（在 drive_item 之后）；

2. read_item 中检测到 error 时不再 return，改为记录 uvm_error 并让控制流正常回到 process_trans；

3. 加入计数器：get_cnt（已取到 item 数）和 done_cnt（已 item_done 数），在每次 item_done 后 assert 两者相等；

4. 错误路径构造 error rsp 回传，便于 sequence 侧感知错误并采取行动。

【完整答案代码】

```
// === 修复前（bug 版，仅供对比）===
// task process_trans_buggy();
// forever begin
// seq_item_port.get_next_item(req);
// if (req.type == READ) begin
// read_item(req);
// if (rsp_err) return; // BUG: 漏了 item_done！sequence hang 于此
// end else
// write_item(req);
// seq_item_port.item_done(rsp); // 错误路径走不到这里
// end
// endtask

// === 修复后：所有路径统一 item_done ===
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int get_cnt = 0;
    int done_cnt = 0;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
```

```
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            get_cnt++;

            // 执行事务（内部检测 error 但不 return）
            drive_item(req);

            // 无论正常/error：统一在这里 item_done
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
            done_cnt++;

            // 回归断言：防止日后再次漏 item_done
            assert(get_cnt == done_cnt) else
                `uvm_fatal("ITEM_MISMATCH",
                    $sformatf("BUG: get_cnt=%0d != done_cnt=%0d",
                              get_cnt, done_cnt))
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    // 修复：检测到 error 时记录 uvm_error 但不 return
```

```
    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        if (v_if.drive_cb.rsp_err) begin
            `uvm_error("DRV_ERR",
                $sformatf("Read error on addr=0x%0h @%0t",
                          trans.addr, $time))
            // 不 return！让控制流回到 process_trans 的 item_done
        end else begin
            trans.data = v_if.drive_cb.data;
        end
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

修复核心：item_done 提到 drive_item 调用之后统一执行，read_item 的 error 路径改为 uvm_error 但不 return。计数器 get_cnt/done_cnt + assert 形成回归防护，日后任何漏 item_done 的 bug 都会在仿真期间立即被捕获。

题目 29 修复：try_next_item(null) 误 item_done

难度：中级

标签：try_next_item、bugfix、性能、计数自检、握手一致性

【题目描述】

修复轮询式 driver 中 try_next_item 返回 null 时误调用 item_done 的 bug，同时优化性能。

Bug 分析：

try_next_item(req) 返回 null 意味着当前没有事务，sequencer 没有授权 driver 操作任何 item。

此时调用 item_done() 会告诉 sequencer"我完成了一个不存在的
item"，破坏内部状态机，导致后续事务丢失或握手错乱。

修复 + 优化：

1. 在 if(req != null) 块内才调用 item_done（最简单、最正确的修复）；

2. @clk 放在 try_next_item 之前（每轮消耗一个时钟，防止 0-time 空转）；

3. req==null 时总线 idle（data=0），不驱动任何无效数据；

4. report_phase 打印 processed_cnt，便于与 sequence 发出数量比对（最小自检）。

【完整答案代码】

```
// === 修复前（bug 版，仅供对比）===
// task process_trans_buggy();
// forever begin
// seq_item_port.try_next_item(req);
// drive_item(req); // req 可能为 null！
// seq_item_port.item_done(); // BUG: null 时也调用，破坏 sequencer 状态机
// end
// endtask

// === 修复后：严格区分 null/非null ===
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int processed_cnt = 0;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
```

```
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req;
        forever begin
            // 修复1：先等时钟，避免 0-time 空转
            @(v_if.drive_mp.drive_cb);
            seq_item_port.try_next_item(req);
            if (req != null) begin
                // 修复2：仅在非 null 时驱动并调用 item_done
                drive_item(req);
                seq_item_port.item_done();
                processed_cnt++;
            end else begin
                // null：保持 idle，绝对不调用 item_done
                v_if.drive_cb.data <= 8'd0;
            end
        end
    endtask

    function void report_phase(uvm_phase phase);
        `uvm_info("DRV_STAT",
            $sformatf("[%s] Total processed items: %0d",
                      get_full_name(), processed_cnt), UVM_NONE)
    endfunction

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        v_if.drive_cb.data <= trans.data;
    endtask
```

```
endclass
```

【模版套用说明】

修复两处：(1) item_done 移到 if(req!=null) 块内；(2) @clk 放在 try_next_item 之前消耗时钟防止 0-time 空转。processed_cnt 在 report_phase 打印，便于与 scoreboard/sequence 发出数量比对进行自检。

题目 30 修复：Pipeline 中对象被重随机化（clone/copy）

难度：高级

标签：pipeline、对象复用、clone/copy、数据一致性、回归复现

【题目描述】

修复 pipeline driver 中由对象复用导致的数据污染 bug。

Bug 根因：

get() 在取到 req 后立即解除 sequence 阻塞，sequence 可以在 driver 还在执行该 req 的 data_phase 期间，

对同一个 req 对象执行 randomize()，覆盖掉 driver 正在读取的字段值（addr/data/type 等），导致数据不一致。

修复方案（最小改动）：

1. get(req) 返回后立即执行 $cast(req_snap, req.clone())，此后所有 drive 操作使用 req_snap；

2. req_snap 是独立的对象，不受 sequence randomize 影响；

3. sequence_id 必须从原始 req 获取（clone 不保证 sequence_id 一致），用于 put_response 路由；

4. 加两处 uvm_info 日志（快照时 + data_phase 完成后），打印关键字段，证明一致性；

5. 不引入任何等待或串行化，pipeline 吞吐完全保留。

【完整答案代码】

```
// === 修复前（bug 版，仅供对比）===
// task process_trans_buggy();
// my_transaction req;
// forever begin
// seq_item_port.get(req);
// // BUG: 没有 clone！sequence 重随机化后 req.data 可能已被改写
// @(v_if.drive_mp.drive_cb);
// v_if.drive_cb.data <= req.data; // 数据已污染！
// end
// endtask

// === 修复后：get 后立即 clone ===
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
```

```
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, req_snap, rsp;
        forever begin
            seq_item_port.get(req);

            // 修复：立即 clone，与原 req 对象完全解耦
            $cast(req_snap, req.clone());

            `uvm_info("DRV",
                $sformatf("SNAP addr=0x%0h data=0x%0h type=%0d",
                          req_snap.addr, req_snap.data, req_snap.type),
                UVM_HIGH)

            // 数据相位：全部使用 req_snap（不受 sequence 重随机化影响）
            drive_item(req_snap);

            `uvm_info("DRV",
                $sformatf("POST addr=0x%0h data=0x%0h (should match SNAP)",
                          req_snap.addr, req_snap.data),
                UVM_HIGH)

            // sequence_id 从原始 req 获取，不能用 req_snap
            $cast(rsp, req_snap.clone());
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.put_response(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
```

```
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

修复一行代码：get() 后加 $cast(req_snap, req.clone())，后续全部操作用 req_snap。sequence_id 必须从原始 req 获取（clone 可能不保留 sequence_id）。SNAP + POST 两条日志可在仿真 log 中 grep 对比，证明数据一致性。

题目 31 Driver 单元测试 Stub：无 DUT 验证 Driver

难度：高级

标签：unit test、sequencer stub、可测性、接口记录、自检

【题目描述】

构建 driver 单元测试框架，在没有 DUT 的情况下独立验证 driver 的握手行为。

框架组成：

1. seq_stub：模拟 sequencer，用 mailbox 实现 get_next_item（阻塞等待 req）和 item_done（记录调用次数）；

2. 测试流程：预填 3 笔写 req（data = 0x01/0x02/0x03），运行若干周期后在 report_phase 检查结果；

3. 自检断言：item_done 次数 == 3，失败时打印期望值 vs 实际值；

4. 固定 seed（+sv_seed）保证可复现，运行时间 < 500 个时钟周期；

5. 单测不依赖完整 env/scoreboard，便于回归集成到 CI 流程。

【完整答案代码】

```
// === sequencer stub：模拟 get_next_item/item_done ===
class seq_stub extends uvm_component;
    mailbox #(my_transaction) req_mb;
    int item_done_cnt = 0;
    `uvm_component_utils(seq_stub)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        req_mb = new(0);
    endfunction

    // driver 调用：阻塞等待下一个 req
    task get_next_item(output my_transaction req);
        req_mb.get(req);
    endtask

    // driver 调用：完成握手（记录次数）
    task item_done(input my_transaction rsp = null);
        item_done_cnt++;
        `uvm_info("STUB",
            $sformatf("item_done #%0d called", item_done_cnt), UVM_HIGH)
    endtask

    function void push_req(my_transaction req);
        req_mb.put(req);
```

```
    endfunction
endclass

// === 最小单元测试 ===
class driver_unit_test extends uvm_test;
    seq_stub sqr_stub;
    my_driver dut_drv;
    `uvm_component_utils(driver_unit_test)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sqr_stub = seq_stub::type_id::create("sqr_stub", this);
        dut_drv = my_driver::type_id::create("dut_drv", this);

        // 预置 3 笔写 req（data = 0x01, 0x02, 0x03）
        begin
            my_transaction req;
            for (int i = 0; i < 3; i++) begin
                req = my_transaction::type_id::create($sformatf("req%0d", i));
                req.type = 1; // write
                req.data = 8'(i + 1); // 0x01, 0x02, 0x03
                sqr_stub.push_req(req);
            end
        end
        uvm_config_db#(seq_stub)::set(this, "dut_drv", "sqr_stub", sqr_stub);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        #3000ns; // 运行 300 个时钟周期（假设 10ns/clk）
        phase.drop_objection(this);
    endtask

    function void report_phase(uvm_phase phase);
        if (sqr_stub.item_done_cnt !== 3)
            `uvm_error("UT_FAIL",
                $sformatf("Expected 3 item_done, got %0d",
                          sqr_stub.item_done_cnt))
        else
            `uvm_info("UT_PASS",
                "item_done count check PASS (3/3)", UVM_NONE)
    endfunction
endclass
// 运行：vsim +sv_seed=42 work.top -do "run -all; quit"
// 检查：grep "UT_PASS" transcript
```

【模版套用说明】

seq_stub 模拟 get_next_item/item_done API，driver 无需修改即可被隔离测试。unit test 预置 3 笔 req，report_phase 检查 item_done 计数和时间约束。固定 seed
保证可复现，便于回归集成到 CI 流程。

题目 32 Driver 内建 Stimulus 覆盖率 Covergroup

难度：中级

标签：functional coverage、covergroup、采样时机、config 开关、reset 豁免

【题目描述】

在 driver 内部实现 stimulus coverage（刺激侧功能覆盖率），反映 driver 实际驱动了哪些刺激组合。

设计要点：

1. covergroup 用 with function sample(my_transaction req) 参数化，避免使用全局采样时钟；

2. 覆盖点设计：cp_cmd（读/写类型）、cp_byte_en（字节使能组合）、cp_addr_range（地址分段）、交叉覆盖；

3. 采样时机：在 get_next_item 成功后（accept 阶段）、drive_item 执行前采样，确保"已决定要发这个刺激"；

4. cov_en 开关：通过 config_db 注入，false 时跳过 sample 调用；

5. reset 豁免：检查 v_if.mon_cb.reset==1 才采样，reset 期间的无效事务不计入覆盖率；

6. 覆盖率命名：set_inst_name 为 driver 层次路径，便于多实例 agent 分别统计。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    bit cov_en = 1;

    covergroup drv_cg with function sample(my_transaction req);
        cp_cmd: coverpoint req.type {
            bins read = {0};
            bins write = {1};
        }
        cp_byte_en: coverpoint req.byte_en {
            bins full = {4'hF};
            bins half_l = {4'h3};
            bins half_h = {4'hC};
            bins single = {4'h1, 4'h2, 4'h4, 4'h8};
            bins other = default;
        }
        cp_addr_range: coverpoint req.addr[31:28] {
            bins low = {[4'h0:4'h3]};
            bins mid = {[4'h4:4'h7]};
            bins high = {[4'h8:4'hF]};
        }
        cx_cmd_x_be: cross cp_cmd, cp_byte_en;
```

```
    endgroup

    `uvm_component_utils(my_driver)

    function new(string name, uvm_component parent);
        super.new(name, parent);
        drv_cg = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
        void'(uvm_config_db#(bit)::get(this, "", "cov_en", cov_en));
        drv_cg.set_inst_name({get_full_name(), ".drv_cg"});
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);

            // 采样时机：accept 阶段（取到 item 后、驱动前）
            // reset 豁免：reset 期间不采样
            if (cov_en && v_if.mon_cb.reset) begin
                drv_cg.sample(req);
                `uvm_info("DRV_COV",
                    $sformatf("Sampled: type=%0d addr=0x%0h be=0x%0h @%0t",
                              req.type, req.addr, req.byte_en, $time),
                    UVM_HIGH)
            end

            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
```

```
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

covergroup 用 with function sample() 参数化，在 process_trans 的 accept 阶段调用（get_next_item 成功后、drive_item 前）。reset 豁免：检查 v_if.mon_cb.reset==1 才采样。set_inst_name 支持多 agent 实例独立统计覆盖率。

题目 33 rsp_port 专用响应通道连接与回压

难度：高级

标签：rsp_port、rsp_export、connect_phase、回压队列、健壮性

【题目描述】

实现带专用响应端口（rsp_port）的 driver，将响应通过独立通道回传而非通过 item_done。

架构设计：

1. driver 使用内建的 rsp_port（uvm_driver#(REQ,RSP) 模板参数中的 RSP 类型端口）；

2. process_trans 中 item_done 不带 rsp 参数，响应通过 rsp_port.write(rsp) 独立发布；

3. agent 的 connect_phase 中：driver.rsp_port -> uvm_tlm_fifo.put_export（提供回压缓冲）-> sequencer.rsp_export；

4. 健壮性：检查 rsp_port 是否已连接（size() > 0），未连接时 uvm_warning 而非 fatal，支持无响应通道的简化场景；

5. uvm_tlm_fifo 深度可配置（本例16），防止 driver 被 sequence 不取响应而阻塞。

【完整答案代码】

```
// === driver：使用专用 rsp_port 通道 ===
class my_driver extends uvm_driver#(my_transaction, my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
```

```
            seq_item_port.get_next_item(req);
            drive_item(req);

            $cast(rsp, req.clone());
            rsp.set_sequence_id(req.get_sequence_id());

            // 通过专用 rsp_port 发布（健壮性：未连接时降级为 warning）
            if (rsp_port.size() > 0)
                rsp_port.write(rsp);
            else
                `uvm_warning("RSP_NC", "rsp_port not connected, response dropped")

            // item_done 不带 rsp（rsp 已通过专用通道发出）
            seq_item_port.item_done();
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass

// === agent：connect_phase 连接 rsp_port（含 TLM FIFO 回压缓冲）===
class my_agent extends uvm_agent;
    my_driver drv;
    my_sequencer sqr;
    uvm_tlm_fifo#(my_transaction) rsp_fifo; // 回压缓冲，深度16
    `uvm_component_utils(my_agent)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (get_is_active() == UVM_ACTIVE) begin
            drv = my_driver::type_id::create("drv", this);
            sqr = my_sequencer::type_id::create("sqr", this);
```

```
            rsp_fifo = new("rsp_fifo", this, 16);
        end
    endfunction

    function void connect_phase(uvm_phase phase);
        if (get_is_active() == UVM_ACTIVE) begin
            drv.seq_item_port.connect(sqr.seq_item_export);
            // rsp 通道：driver.rsp_port -> FIFO -> sequencer.rsp_export
            drv.rsp_port.connect(rsp_fifo.put_export);
            sqr.rsp_export.connect(rsp_fifo.get_peek_export);
        end
    endfunction
endclass
```

【模版套用说明】

rsp_port.write() 替代 item_done(rsp)，item_done 不带参数。TLM FIFO 在 agent 层提供回压缓冲（深度16），driver 不会因 sequence 不取响应而阻塞。rsp_port.size() 检查连接状态，未连接时 warning 而非 fatal 保证健壮性。

题目 34 IEEE 1800.2 迁移：去除 Deprecated API

难度：高级

标签：1800.2、deprecated API、迁移、编译矩阵、回归验证

【题目描述】

将 UVM 1.2 风格的 driver 迁移到 IEEE 1800.2 标准 API，去除所有 deprecated 依赖。

常见 deprecated API 及对应修复方案：

1. uvm_update_sequence_lib 宏：1800.2 中已完全移除，直接删除；

2. begin_tr(tr) 无保留返回值：改为 void'(begin_tr(tr))，明确丢弃返回值；

3. set_id_info(rsp, req)：替换为 rsp.set_sequence_id(req.get_sequence_id())；

4. ovm_* 前缀（OVM 遗留）：全部改为 uvm_* 前缀；

5. 编译矩阵：提供 UVM 1.2 兼容模式和 IEEE 1800.2
严格模式两种编译命令，以及关键编译开关的作用说明。

【完整答案代码】

```
// === deprecated API 对照表 ===
// UVM 1.2 (deprecated) | IEEE 1800.2 替换
// -----------------------------|-----------------------------
// `uvm_update_sequence_lib | 删除（1800.2 已移除）
// begin_tr(tr) | void'(begin_tr(tr))
// set_id_info(rsp, req) | rsp.set_sequence_id(req.get_sequence_id())
// ovm_component | uvm_component
// ovm_transaction | uvm_sequence_item

// === 修复后：完全兼容 IEEE 1800.2 的 driver ===
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
```

```
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);

            // 修复1：void'(begin_tr(...)) 明确丢弃返回值（1800.2 要求）
            void'(begin_tr(req));

            drive_item(req);
            end_tr(req);

            // 修复2：set_sequence_id 替代 deprecated set_id_info
            $cast(rsp, req.clone());
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass

// === 编译矩阵说明 ===
// 方式1：UVM 1.2 兼容模式（含 deprecated API 支持）
// vlog +incdir+${UVM_HOME}/src ${UVM_HOME}/src/uvm_pkg.sv
// +define+UVM_ENABLE_DEPRECATED_API my_driver.sv
//
// 方式2：IEEE 1800.2 严格模式（推荐，不启用 deprecated）
```

```
// vlog +incdir+${UVM_IEEE_HOME}/src ${UVM_IEEE_HOME}/src/uvm_pkg.sv
// my_driver.sv
//
// 回归验证：
// vsim -sv_seed 42 work.top -do "run -all; quit"
// grep "UVM_ERROR" transcript # 期望：0 errors
```

【模版套用说明】

核心修复两处：(1) void'(begin_tr(req)) 消除未使用返回值警告/错误；(2) set_sequence_id 替代 set_id_info。模版结构本身已是 1800.2
兼容形式，不需要大改。编译矩阵注释说明两种编译方式的差异，便于 CI 验证。
